

JENKINS CI/CD PIPELINE

Build, Test, Docker Build & Docker Push

Technical Deliverables Report

Project: Sportswear Store CI/CD Pipeline

Platform: Jenkins on Windows

Repository: dhuor2001-debug/sportswear-store

Prepared by: Abraham Makur Dhuor

Date: September 2026

1. Executive Summary

This report documents the CI/CD pipeline work completed for the Sportswear Store application. The pipeline uses Jenkins to automate source checkout, application build, automated testing, Docker image creation, and Docker Hub image publication. During testing, an initial startup timing problem was diagnosed and resolved by replacing a blind one-second wait with readiness polling and by exposing the spawned server's standard output and error streams.

The final Jenkins build completed successfully across all implemented stages, including Docker Push. The evidence required for submission should be inserted into the marked screenshot areas throughout this report.

2. Deliverables Checklist

#	Deliverable	Evidence to Include	Status
1	Jenkins pipeline screenshot	Jenkins Stages/Pipeline view showing the completed stages	Completed
2	Successful build screenshot	Build result with all stages green	Completed
3	Docker image screenshot	Docker Build log showing image name/tag(s)	Completed
4	Jenkinsfile	Pipeline source file or screenshot/code appendix	To attach/paste
5	GitHub repository link	Repository URL	Available
6	Test evidence	Successful health check and products endpoint output	Completed
7	Technical explanation	Description of each pipeline stage	Included
8	Friday deployment activity	Deploy → Verify → Break → Rollback evidence	Planned / to complete
9	Architecture/flow diagram	GitHub → Jenkins → Build → Test → Docker → Deploy → Verify → Rollback	To insert

3. Project Overview

The project demonstrates a basic CI/CD workflow for a Node.js sportswear-store application. Jenkins is used as the automation server, while Docker is used to package the application into a deployable image. The pipeline is designed to provide repeatable validation before an image is pushed to Docker Hub.

3.1 Technology Stack

- Application: Node.js
- CI/CD: Jenkins
- Containerization: Docker

- Container Registry: Docker Hub
- Source Control: Git/GitHub
- Operating environment: Windows
- Testing: Node.js HTTP test script

4. CI/CD Pipeline Workflow

The implemented Wednesday workflow is: Checkout → Build → Test → Docker Build → Docker Push. The later deployment activity is planned as: Deploy → Verify → Break → Rollback.

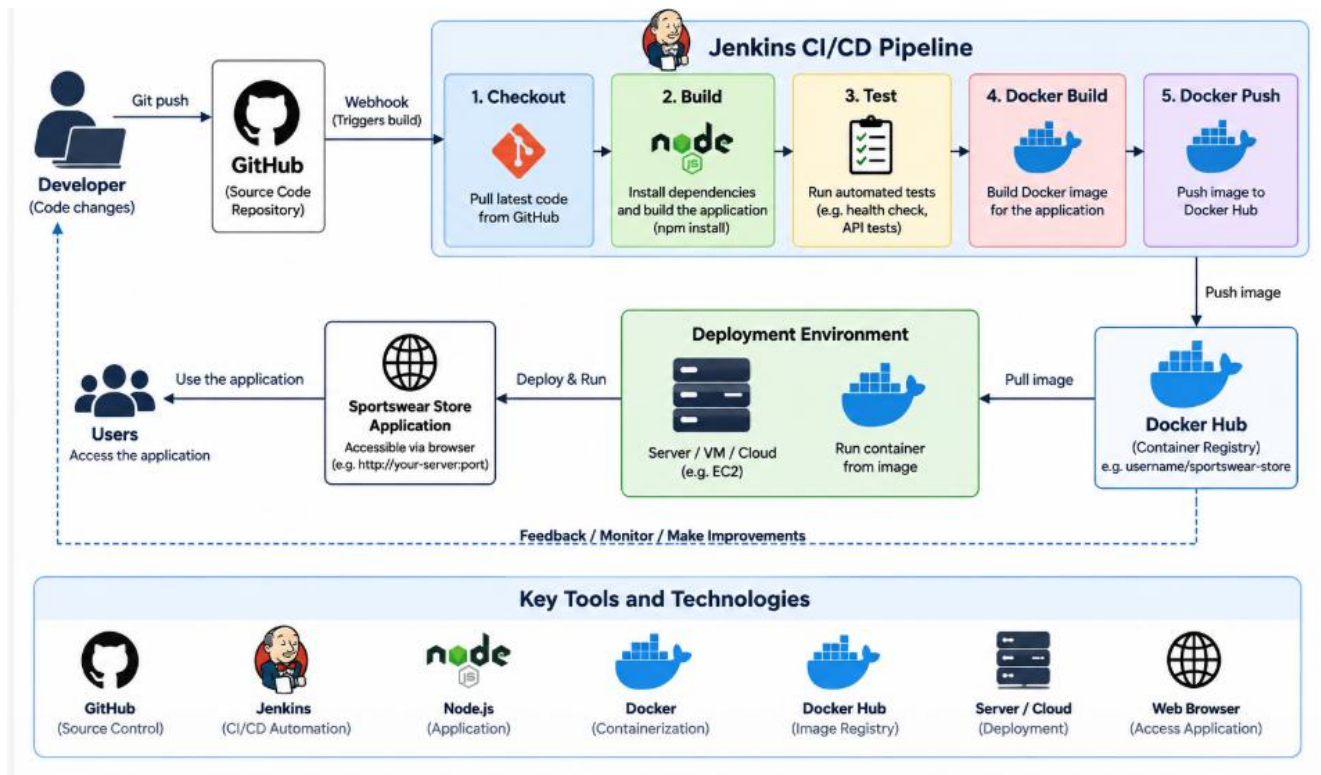


Figure 1: Overall CI/CD pipeline and deployment workflow.

5. Pipeline Stages and Their Functions

5.1 Checkout

Jenkins retrieves the project source code from the GitHub repository so that the pipeline operates on the latest committed version.

Evidence: Jenkins stage view or console log showing successful source checkout.

5.2 Build

The application dependencies and build requirements are processed so the application can be prepared for testing and containerization.

Evidence: Jenkins Build stage log.

5.3 Test

The application is started on a test port and the automated test script checks the /health endpoint and the /api/products endpoint. The successful run reported a 200 health response and six products returned by the products endpoint.

Evidence: console output showing “Health check passed” and “Products endpoint returned 6 items”.

5.4 Docker Build

The application is packaged into a Docker image. The image is tagged so it can be identified and published to the container registry.

Evidence: Docker Build console log showing the generated image tag(s).

5.5 Docker Push

The successfully built image is authenticated and pushed to Docker Hub using the configured Jenkins credential.

Evidence: Jenkins stage view and/or console log showing successful Docker Push.

6. Automated Test Reliability and Troubleshooting

The first version of test.js attempted to contact the application after a fixed one-second delay. This produced ECONNREFUSED when the server had not yet finished starting. The improved test script was designed to expose server stdout/stderr and repeatedly poll the /health endpoint until the server became ready.

The updated test script uses `child_process.spawn()`, captures server output, and retries the health request. After the server is ready, it verifies that /health returns HTTP 200 and that /api/products returns a non-empty JSON array.

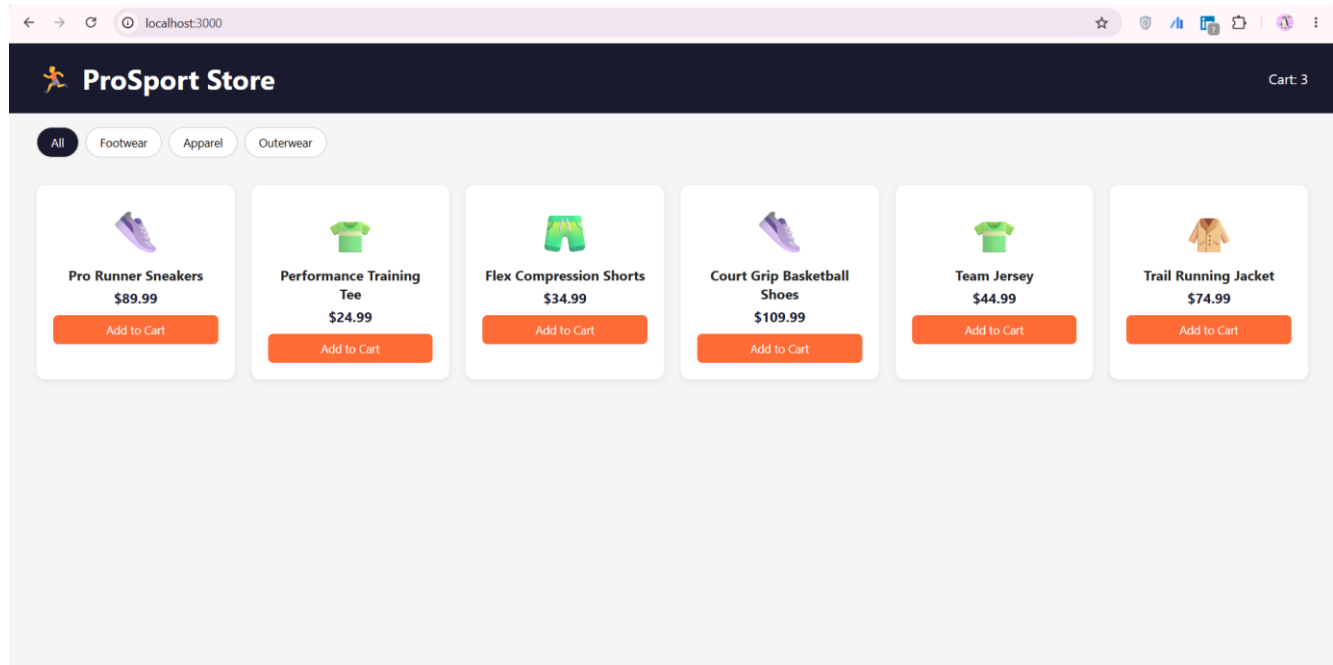


Figure 2: Successful automated test execution.

6.1 Initial Failure and Root Cause

An initial run returned ECONNREFUSED on localhost:3999. This indicated that the spawned server process was not listening when the test attempted the connection. After improving the diagnostic output, a separate local run exposed

the underlying dependency issue: “Cannot find module 'express’”. The local project environment did not have its Node.js dependencies installed.

Running npm install restored the required dependencies. The retry-based test was then used to validate server readiness before executing the endpoint checks.

7. Jenkins Build Evidence

The final pipeline execution was reported as fully green, with every implemented stage passing, including Docker Push. The successful run therefore provides the primary evidence that the Wednesday CI/CD deliverable was completed end-to-end.

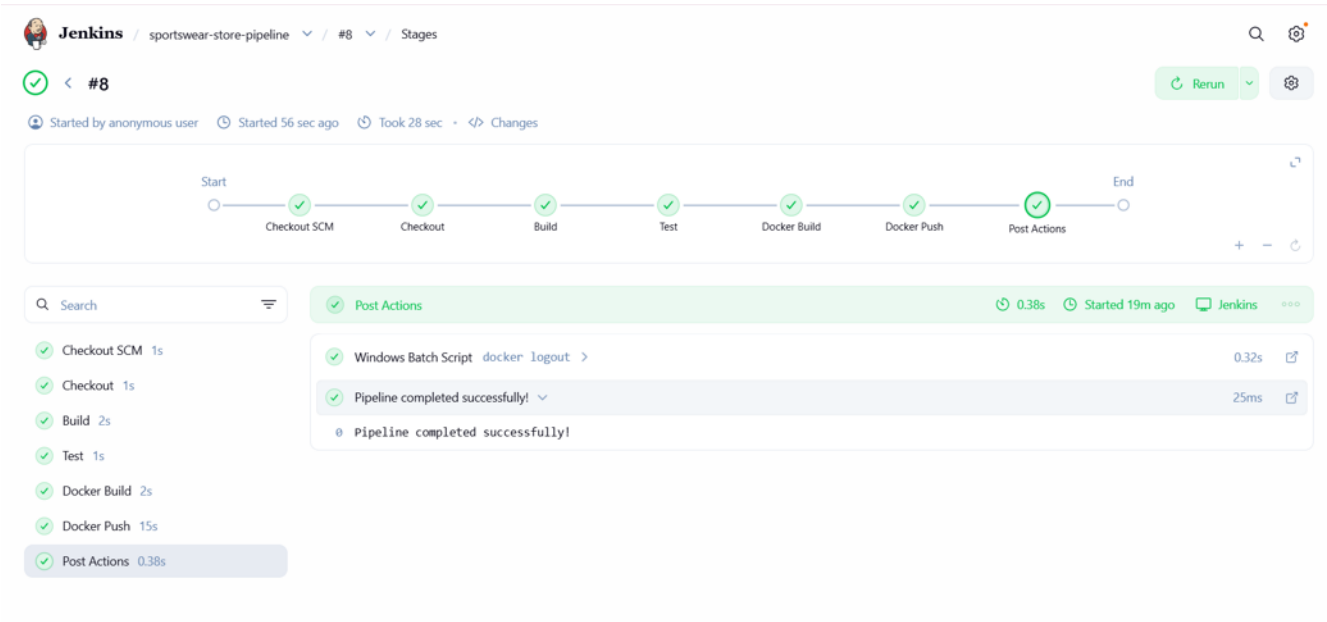


Figure 3: Jenkins pipeline showing successful completion of all implemented stages.

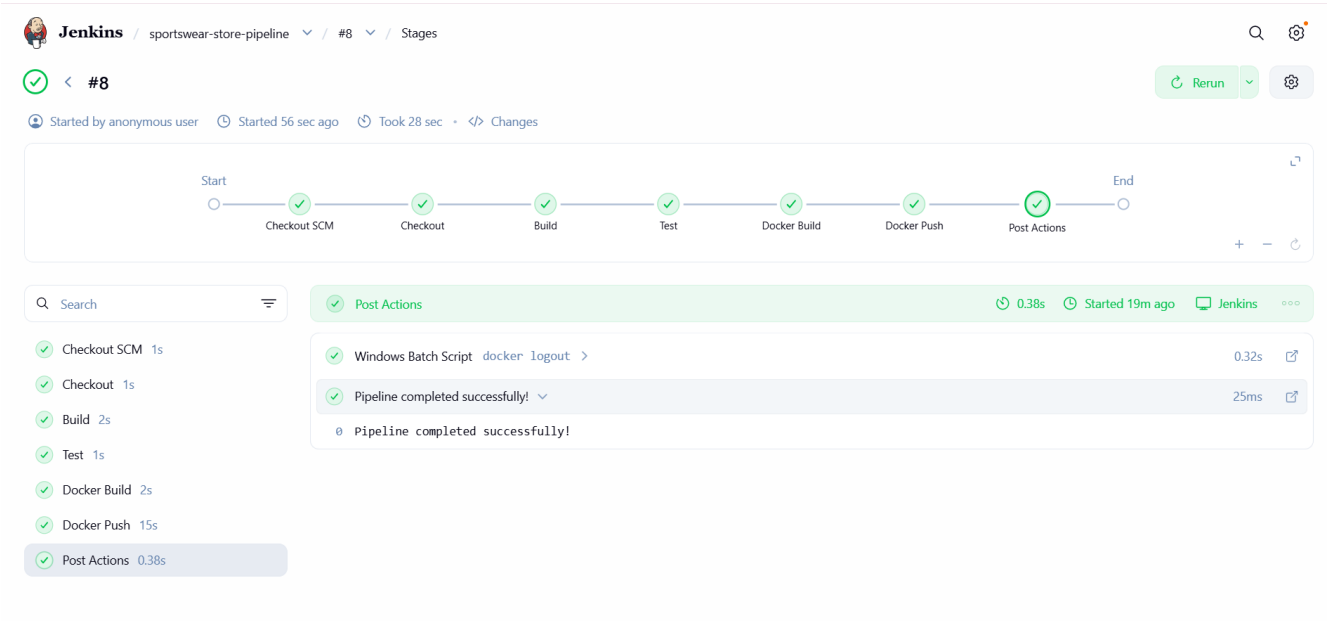


Figure 4: Successful Jenkins build.

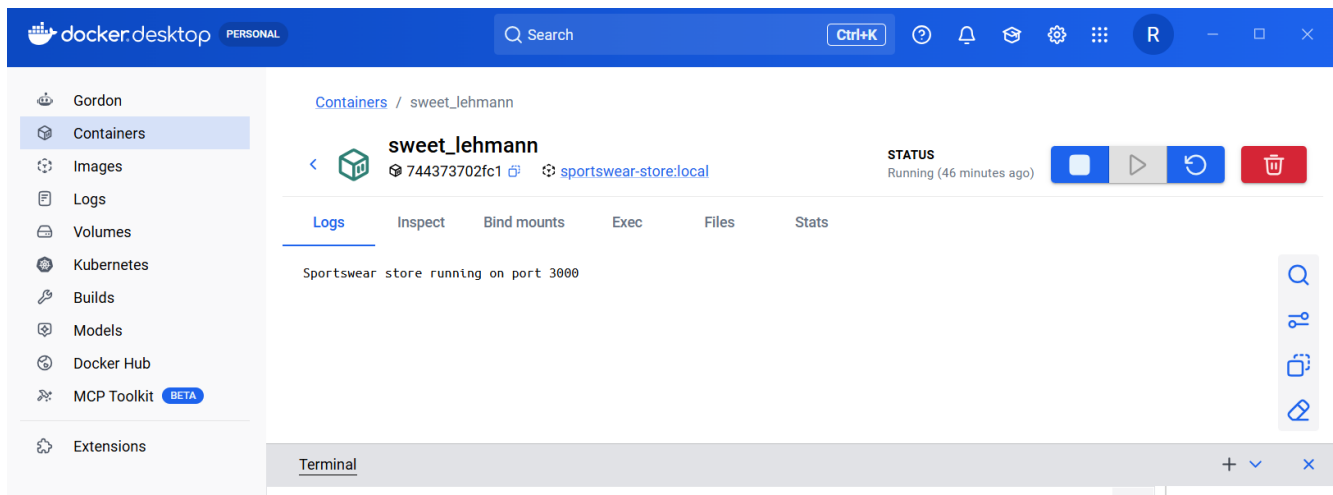


Figure 5: Docker image build and tagging evidence.

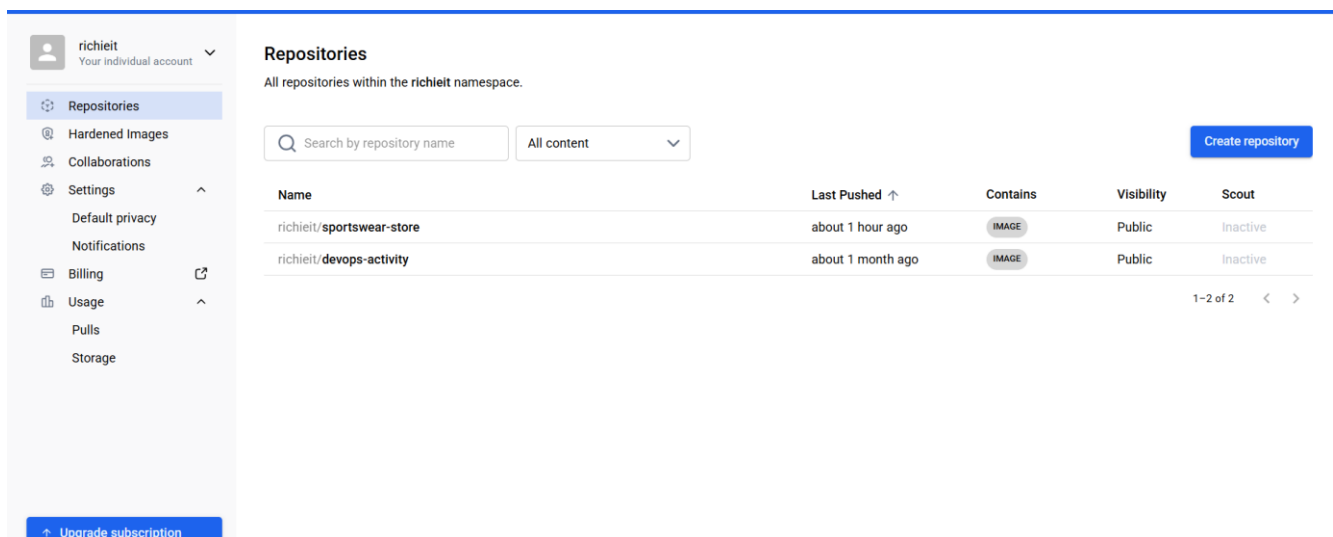


Figure 6: Docker image push evidence.

8. Jenkinsfile Deliverable

The Jenkinsfile defines the pipeline stages and commands executed by Jenkins. Include the final Jenkinsfile in the submission either as formatted code in an appendix or as a screenshot of the file in the project repository.

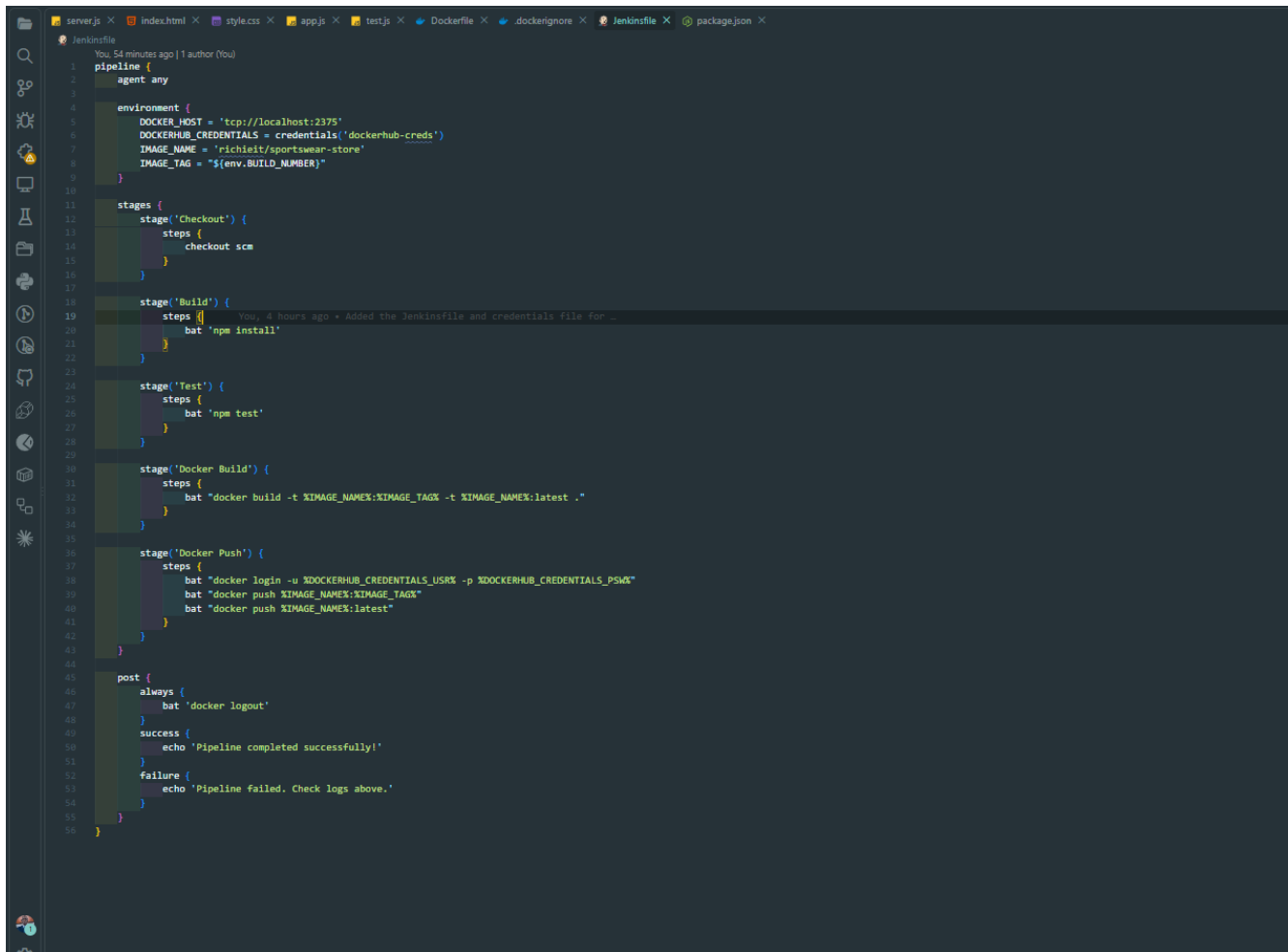


Figure 7: Jenkinsfile used for the CI/CD pipeline.

9. Source Code and Repository

GitHub repository:

<https://github.com/dhuor2001-debug/sportswear-store>

The repository should contain the application source, Jenkinsfile, test script, package files, Docker configuration, and any other files required to reproduce the pipeline.

Recommended evidence: paste a screenshot of the repository root showing the Jenkinsfile, application files, test.js, package.json, and Docker-related files.

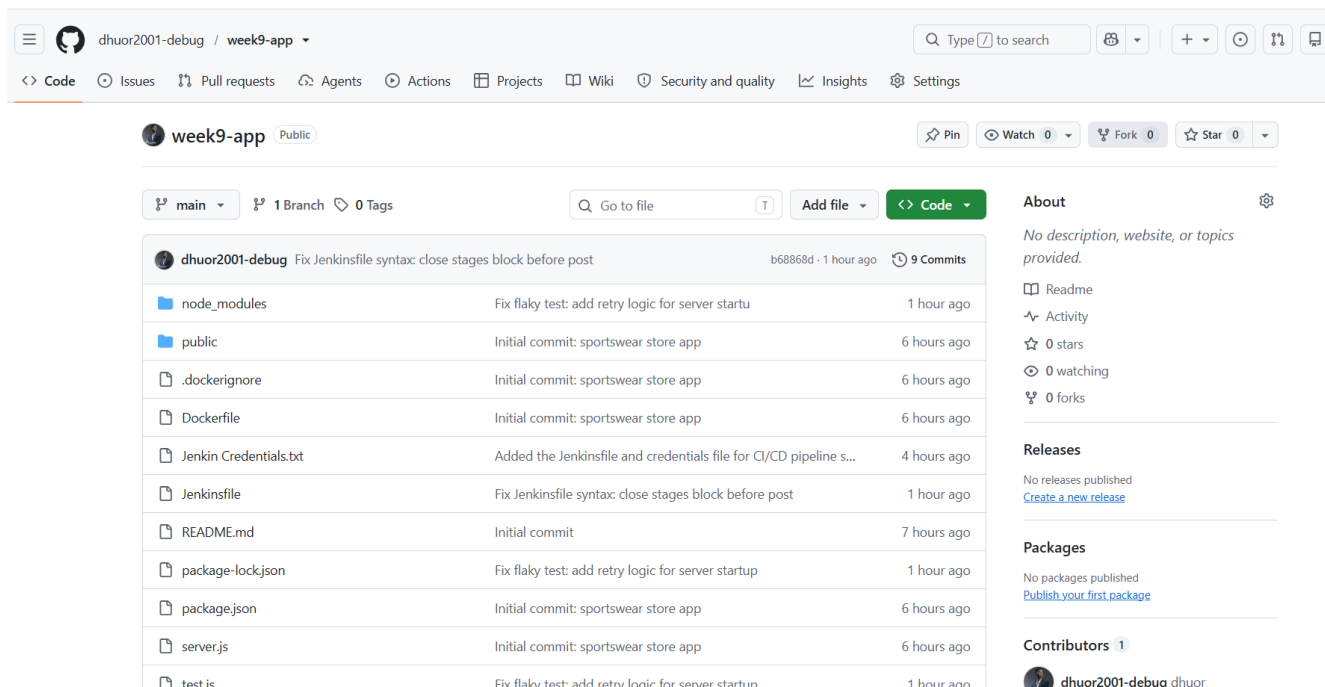


Figure 8: GitHub repository contents.

10. Docker Image and Registry Evidence

The pipeline's Docker stages convert the application into a container image and publish it to Docker Hub. The Docker Push stage depends on the Jenkins credential configured for Docker Hub authentication.

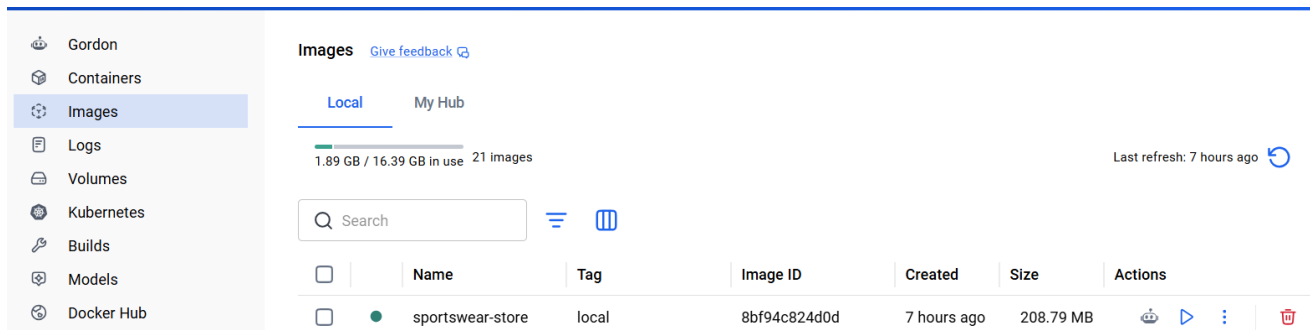


Figure 9: Docker Hub image repository.

11. Conclusion

The implemented Jenkins pipeline demonstrates an automated CI/CD workflow for the Sportswear Store application. The pipeline successfully progressed through source checkout, build, automated testing, Docker image creation, and Docker image publication. The test process was also improved to handle application startup timing and to expose useful server-side errors during failures.

The remaining deployment deliverable extends this foundation by demonstrating deployment verification, controlled failure, and rollback. Once the marked screenshots and the final Jenkinsfile/repository evidence are inserted, this document can serve as the consolidated technical submission report.